

1. Conceitual – Importância da Preparação

A etapa de preparação de dados é frequentemente mais decisiva para a performance de um modelo de Machine Learning do que a escolha do algoritmo em si. Isso acontece porque todos os algoritmos aprendem padrões a partir dos dados que recebem e se esses dados forem inconsistentes, incompletos, desbalanceados ou pouco volume, o modelo não conseguirá generalizar bem, independentemente de sua complexidade. Por exemplo, variáveis em escalas diferentes podem prejudicar algoritmos baseados em distância, como KNN, enquanto dados ausentes ou categóricos não tratados podem gerar vieses ou erros de predição. Um modelo simples, treinado com dados limpos e bem preparados, muitas vezes supera um modelo complexo que recebeu dados sujos.

2. Valores Ausentes

Quando um dataset apresenta valores ausentes, existem várias formas de lidar com o problema. Uma abordagem é remover registros incompletos, mas essa estratégia é adequada apenas quando a proporção de valores ausentes é pequena, pois remover muitas linhas pode levar à perda significativa de informação e assertividade. Alternativamente, podemos substituir os valores ausentes pela média ou mediana da variável, o que é simples e funciona bem quando não há muitos outliers. Outra possibilidade é usar imputação baseada em modelos preditivos, onde outras variáveis do dataset ajudam a estimar o valor ausente. Para variáveis categóricas, pode-se criar uma categoria “desconhecido” para não perder os registros. A escolha da técnica depende da proporção de dados ausentes e da importância da variável para o modelo.

3. Normalização vs Padronização

A normalização e a padronização são técnicas de escalonamento de variáveis que ajustam a distribuição dos dados. A normalização Min-Max transforma os valores para um intervalo fixo, geralmente entre 0 e 1, mantendo a proporção entre eles, e é recomendada em algoritmos sensíveis à magnitude, como redes neurais ou KNN. A padronização Z-score, por sua vez, centraliza os dados em torno de zero e ajusta pelo desvio padrão, o que é útil para algoritmos que assumem distribuição normal ou dependem da variância, como regressão linear e SVM. Enquanto a normalização pode ser afetada por outliers, a padronização lida melhor com valores extremos.

4. Dados Categóricos

Variáveis categóricas, como “clima” com valores sol, chuva e nublado, não podem ser usadas diretamente em modelos que operam apenas com números. Uma abordagem comum é a codificação One-Hot, que cria uma coluna para cada categoria e atribui 1 ou 0 dependendo da ocorrência. Se utilizarmos apenas codificação numérica simples, como 0, 1 e 2, o modelo pode interpretar uma ordem inexistente, assumindo, por exemplo, que nublado é maior que chuva, o que não faz sentido.

5. Overfitting e Separação de Dados

Dividimos os dados em conjuntos de treino, validação e teste para evitar o overfitting e garantir que o modelo consiga generalizar bem. O conjunto de treino é usado para aprendizado do modelo, o de validação serve para escolher hiperparâmetros e monitorar a performance durante o desenvolvimento, enquanto o teste avalia a capacidade do modelo em dados nunca vistos. Utilizar o teste durante o treino é problemático porque o modelo pode “memorizar” os dados e apresentar métricas artificialmente altas, comprometendo sua aplicabilidade em cenários reais.

6. Balanceamento de Classes

Quando as classes estão desbalanceadas, como 95% de um grupo e 5% de outro, o modelo tende a favorecer a classe majoritária, ignorando a minoritária. Isso pode gerar alta acurácia aparente, mas baixa capacidade de identificar casos raros. Para contornar isso, é possível aumentar artificialmente a classe minoritária (oversampling), reduzir a quantidade de registros da classe majoritária (undersampling) ou ajustar os pesos das classes na função de perda para penalizar mais os erros da classe minoritária.

7. Detecção de Outliers

Outliers são valores extremos que se distanciam significativamente da distribuição normal dos dados. Eles podem representar erros de medição ou casos raros importantes. Removê-los sem cuidado pode prejudicar o modelo, principalmente quando esses pontos são legítimos e carregam informação relevante. Uma forma de identificá-los é utilizando métodos estatísticos, como o IQR, que considera outliers os valores que estão além de 1,5 vezes o intervalo interquartilico.

8. Feature Engineering

Transformar variáveis pode melhorar a performance do modelo. Por exemplo, em vez de usar diretamente a data de nascimento, calcular a idade permite ao modelo capturar padrões mais claros e reduz a complexidade do dado, tornando o aprendizado mais eficiente e interpretável. Além disso, é mais fácil trabalhar com idade em análise de risco de crédito ou segmentação etária.

9. Pipeline de Pré-processamento

Um pipeline completo começa tratando valores ausentes, imputando médias ou categorias “desconhecidas”, seguido da codificação de variáveis categóricas. Em seguida, aplica-se escalonamento, seja normalização ou padronização, e são tratados outliers quando necessário. Por fim, realiza-se o balanceamento de classes, antes de dividir os dados em treino, validação e teste.

10. Estudo de Caso – Crédito Bancário

No caso de previsão de aprovação de crédito, o primeiro passo é lidar com valores ausentes nas variáveis numéricas, como renda e idade, substituindo por mediana, e para categóricas, como estado civil, usando a categoria “desconhecido”. Em seguida, as variáveis categóricas são codificadas, por exemplo, via One-Hot. É interessante criar variáveis derivadas, como faixa etária, para melhorar a representação. As variáveis numéricas devem ser escaladas para uniformizar a influência no modelo, e outliers extremos precisam ser avaliados cuidadosamente. Se houver desbalanceamento entre aprovados e reprovados, técnicas como SMOTE ou ajuste de pesos podem ser aplicadas. Por fim, divide-se o dataset em treino, validação e teste, garantindo que todas as etapas de pré-processamento sejam aplicadas dentro de um pipeline reproduzível, pronto para treinar o modelo de forma robusta.